Architecture and block diagrams

Dubai Real Estate CRM · Document 02 · Version 1.2 · 11 October 2026 · Matches SRS v1.2 (adds subscriptions, billing and owner console)

---

1. The system at a glance

Users open one web app on computer or phone. Every request goes through Cloudflare, which forwards it to one of two identical app servers in the AWS UAE region. Both servers share one PostgreSQL database that has a live standby copy. Outside services (Meta, WhatsApp, Property Finder, Bayut, Dubizzle, the general webhook, Stripe for subscription payments) send data in through webhooks and receive calls from our background worker. Our own team runs the business from a separate owner console with its own login.

USERS

- **Web browser** — Director, managers, agents, admin, finance
- **Phone (installable web app)** — Agents on the move · attendance camera
- **Clients** — CRM booking page · WhatsApp · email
- **Outside systems** — Send leads and status updates by webhook

↓ HTTPS only

CLOUDFLARE (FREE PLAN) · EDGE

- **DNS + SSL** — crm.example.ae
- **Firewall + DDoS** — Blocks attacks, rate limits login and webhooks
- **CDN cache** — App files, images
- **Cloudflare Tunnel** — Sends traffic to a healthy server; servers have no open ports
- **Health checks** — Stops sending to a failed server

↓ Encrypted tunnel (outbound from each server)

AWS · MIDDLE EAST (UAE) REGION · ME-CENTRAL-1 · ONE VPC

DATA CENTRE A (ZONE 1)

App server A · EC2 t4g.small (2 vCPU, 2 GB) · Docker

- **web** — Next.js screens
- **api** — NestJS REST API
- **worker** — Background jobs
- **gotenberg** — PDF maker
- **cloudflared** — Tunnel

DATA CENTRE B (ZONE 2)

App server B · identical copy

- **web** — Next.js screens
- **api** — NestJS REST API
- **worker** — Background jobs
- **gotenberg** — PDF maker
- **cloudflared** — Tunnel

↓ SQL over TLS · private network

DATABASE · AMAZON RDS POSTGRESQL 16 · MULTI-AZ

- **Primary (zone 1)** — All reads and writes · job queue tables (pg-boss) · row-level security per tenant
- **Standby (zone 2)** — Live copy · takes over automatically in 1–2 min

Automated backups · point-in-time restore 14 days · encrypted

AWS SERVICES USED BY THE SERVERS

- **S3** — Files: brochures, floor plans, invoice PDFs, attendance photos, exports. Private, encrypted.
- **Cognito** — Login + two-factor (authenticator app). Issues tokens checked by api.
- **SES** — Send email · receive portal lead emails
- **SSM Parameter Store** — Secrets and integration tokens
- **CloudWatch** — Logs, metrics, alarms
- **ECR** — Docker images for deployment

OUTSIDE SERVICES · CONNECTED BY THE WORKER AND WEBHOOKS

- **Meta** — Lead ads, spend
- **WhatsApp** — Cloud API, direct with Meta
- **Google Ads** — Leads, spend
- **LinkedIn · TikTok · Snapchat** — Lead forms
- **Property Finder · Bayut · Dubizzle** — Portal leads
- **General webhook** — Website, Google Sheets, Excel, any tool
- **Google Workspace** — Calendar sync, Meet links
- **Microsoft 365** — Calendar, Teams links, mail
- **Gmail** — Agent mailbox
- **AWS SES** — System email
- **Ad spend APIs** — Meta, Google, others
- **Firebase** — Push notifications
- **Stripe** — Subscription cards, invoices, payment webhooks

BUILD AND MONITORING

- **GitHub** — Code + Actions CI/CD
- **Terraform** — Infrastructure as code
- **Sentry** — Error tracking
- **Better Stack** — Uptime checks

Users · Cloudflare edge · AWS UAE · Database · AWS managed services · Outside services · Build and monitoring

Figure 1. Complete system. All personal and financial data stays in the AWS UAE region.

2. What runs on each app server

Both servers run the same five Docker containers. The servers are interchangeable: any request can go to either one, because nothing is stored on the server itself.

CONTAINER	TECHNOLOGY	WHAT IT DOES	TALKS TO
web	Next.js 15, React, TypeScript	Serves all screens. Installable on phones (PWA). Calls the api; never the database directly.	api
api	NestJS, Node.js 22, TypeScript	REST API at `/api/v1`. Checks login token, tenant and permission on every request. Receives webhooks at `/webhooks/*`. Publishes Swagger docs.	PostgreSQL, S3, Cognito, SES
worker	NestJS (same code, worker mode)	Runs background jobs from the pg-boss queue: process incoming leads, send messages, reminders, the 10-minute rule, syncs, exports, dashboard refresh.	PostgreSQL, S3, outside services, gotenberg
gotenberg	Gotenberg 8	Turns invoice and purchase-order HTML into PDF. Internal only.	worker
cloudflared	Cloudflare Tunnel client	Opens an outbound tunnel to Cloudflare so no inbound port is open on the server.	web, api

Jobs are claimed from the database queue with row locking, so running the worker on both servers never processes the same job twice.

3. Inside the backend: modules

The api and worker are one NestJS codebase split into modules. Each module owns its own tables and exposes a service to other modules. A module never reads or writes another module's tables directly.

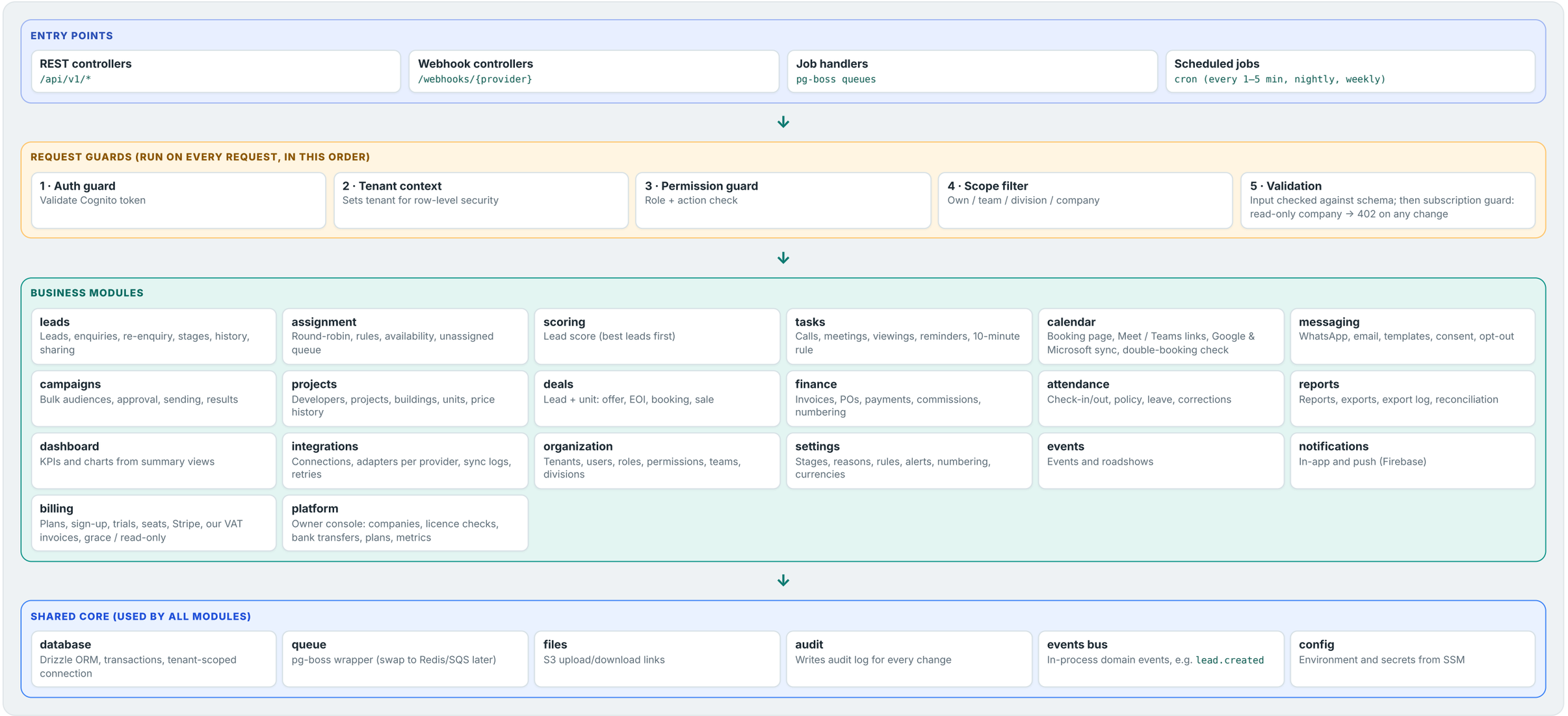


Figure 2. Backend layers. Modules talk to each other through services and domain events, never through each other's tables.

Domain events between modules

EVENT	PUBLISHED BY	HANDLED BY
`lead.created`	leads	assignment (assign owner), scoring (first score), notifications
`lead.re_enquired`	leads	notifications ("Re-enquired" to the same agent), scoring, dashboard
`lead.no_response`	tasks	assignment (next agent or Admin), notifications
`lead.shared`	leads	notifications, audit
`lead.assigned`	assignment	tasks (start 10-minute timer), notifications
`lead.stage_changed`	leads	scoring, dashboard, audit
`message.received`	messaging	leads (timeline), scoring, tasks (stops the 10-minute timer), notifications

`meeting.booked / changed / cancelled`	calendar	leads (timeline, stage), tasks, messaging (confirmation)
`deal.updated`	deals	projects (unit status), leads (timeline)
`invoice.approved / payment.recorded`	finance	leads (timeline), dashboard
`leave.approved`	attendance	assignment (skip on leave days)
`user.receives_leads_changed`	organization	assignment (include / skip user)
`tenant.signed_up`	billing	organization (create company, Admin, default roles and settings), notifications (welcome email)
`subscription.status_changed`	billing	organization (read-only / active), notifications (banner, emails), platform (metrics)
`user.activated`	organization	billing (seat count; the database also blocks going over the paid seats)

4. How a new lead flows through the system

Example: a person fills a Meta lead form. The same path is used for every source; only the adapter in step 3 changes.

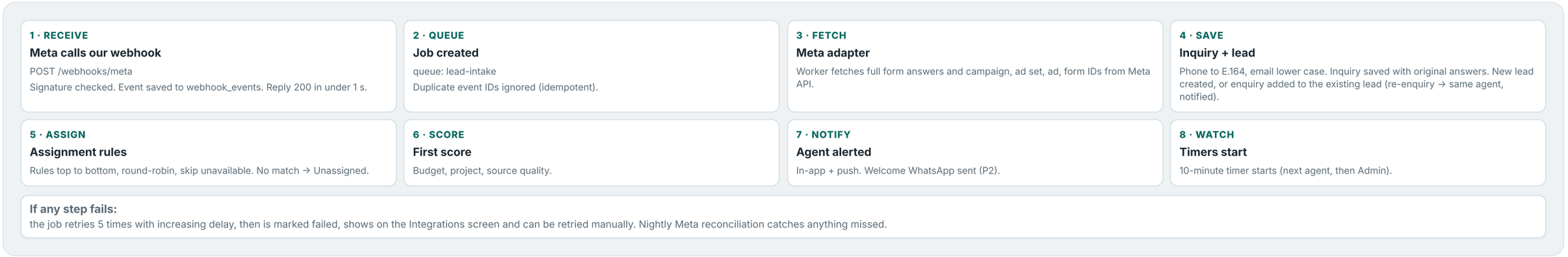


Figure 3. Lead intake. Target: lead visible in the CRM within 1 minute.

5. Background jobs

JOB	WHEN	WHAT IT DOES
lead-intake	On each webhook / import row	Steps 3–7 of Figure 3
send-message	On send	Sends WhatsApp or email, stores provider message ID
message-status	On provider webhook	Updates sent / delivered / read / failed / replied
campaign-dispatch	At scheduled time	Sends approved bulk campaign in batches, skips opted-out
reminders	Every minute	Task reminders and overdue alerts
response-timer	Every minute	10-minute rule: lead not acted on moves to the next available agent in the team; when the whole team misses it, it goes back to the Admin

calendar-sync	Every 5 minutes + webhooks	Google / Microsoft calendar sync
ad-spend-import	Nightly	Imports daily spend from Meta and other ad accounts
meta-reconciliation	Nightly	Compares Meta lead IDs with CRM, re-imports missing
dashboard-refresh	Every 5 minutes	Refreshes dashboard summary views
invoice-overdue	Nightly	Marks unpaid invoices past due date as Overdue
attendance-close-day	Nightly	Marks absent users, applies leave and holidays
report-export	On request (large exports)	Builds Excel/CSV, saves to S3, notifies user
stripe-events	On Stripe webhook	Card saved, invoice paid, payment failed: updates subscription, invoice and payment
billing-dunning	Hourly	Trial ending reminders; payment-failed reminders; after 14 days' grace sets the company read-only
billing-invoices	Daily	Issues our VAT invoices for renewals and extra seats (bank-transfer customers); stores the PDF
tenant-usage	Nightly	Daily usage per company for the owner console
tenant-data-deletion	Nightly	Deletes a cancelled company's data 90 days after the end (export offered first)

6. Security design

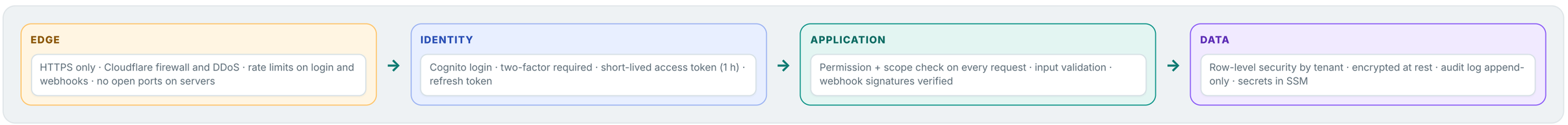


Figure 4. Four layers of protection. A bug in one layer is still stopped by the next.

1. **Tenant isolation:** every business table has `tenant_id`. The api opens each transaction with the user's tenant, and PostgreSQL row-level security only returns that tenant's rows.
2. **Files:** S3 is private. Users get time-limited download links (5 minutes) only for files they are allowed to see.
3. **Owner console:** separate Cognito user pool with two-factor login and a separate database role (`crm_platform`). It can see plans, seats, payments and usage, but no screen or API shows a company's leads or messages. Every action is logged.
4. **Payments:** card details go straight from the browser to Stripe; our servers only keep Stripe IDs and the last 4 digits.
5. **Integration tokens** (Meta, WhatsApp, portals, Google, Microsoft…) are stored encrypted in SSM Parameter Store, never in the database or code.
6. **Webhooks** are accepted only with a valid provider signature or shared secret.

7. Technology choices

LAYER	CHOICE	WHY
Screens	Next.js 15, React, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, Apache ECharts	One codebase for desktop and phone; fast, typed, good charts
Backend	NestJS on Node.js 22, TypeScript	Clear module structure; same language as the screens

Database	PostgreSQL 16 (Amazon RDS), Drizzle ORM, SQL migrations	Reliable; row-level security; full-text and fuzzy search; phone/email matching for re-enquiries
Jobs	pg-boss (queue inside PostgreSQL)	No extra server; switchable to Redis or SQS later
Login	Amazon Cognito (OIDC)	Free up to 10,000 users; standard protocol, switchable to Keycloak
Files	Amazon S3	Cheap, private, encrypted
Email	Amazon SES	Low cost; inbound parsing for portal lead emails
PDF / Excel	Gotenberg, ExcelJS	Open source, run on our servers
Edge	Cloudflare Free + Tunnel	Security, SSL, failover between servers at no cost
Delivery	Docker, GitHub Actions, Amazon ECR, Terraform	Repeatable builds and environments
Monitoring	CloudWatch, Sentry, Better Stack	Logs, errors and uptime alerts

8. Environments

	STAGING (TEST)	PRODUCTION (LIVE)
Servers	1 × t4g.small, office hours only	2 × t4g.small in two zones
Database	PostgreSQL in Docker on the same server	RDS PostgreSQL Multi-AZ
Data	Test data only	Real data
Address	`staging.crm.example.ae`	`crm.example.ae`
Deploy	Automatic on merge to `main`	Manual approval after staging passes

9. How it grows (no rewrite)

PART	TODAY	WHEN IT NEEDS MORE
App servers	2 × t4g.small	Larger instances, then Auto Scaling 2–6 servers
Database	db.t4g.medium Multi-AZ	Larger instance, read replica for reports; self-managed PostgreSQL at very large scale
Job queue	pg-boss	Redis or Amazon SQS (only the queue wrapper changes)
Search	PostgreSQL full-text	OpenSearch at millions of leads
Public images	S3	Cloudflare R2 (no viewing fees)
Login	Cognito	Keycloak above 10,000 active users